

OBSERVATION TASK 6

Title

Collection Framework – Method Implementation – Part 2

Objective / Aim

To study and implement the important methods of various **Java Collection Framework classes** such as `ArrayList`, `LinkedList`, `Vector`, `Stack`, `HashSet`, `LinkedHashSet`, `TreeSet`, `PriorityQueue`, `ArrayDeque`, `HashMap`, `LinkedHashMap`, `TreeMap`, and `Hashtable`.

The program demonstrates operations such as **insertion, deletion, searching, retrieval, updating, sorting, queue operations, stack operations, set operations, and key-value mapping**.

Problem Statement

Develop a Java program to demonstrate the important methods provided by different classes of the Java Collection Framework.

The program should implement and demonstrate:

- `ArrayList`
- `LinkedList`
- `Vector`
- `Stack`
- `HashSet`
- `LinkedHashSet`
- `TreeSet`
- `PriorityQueue`
- `ArrayDeque`
- `HashMap`
- `LinkedHashMap`
- `TreeMap`
- `Hashtable`

The result of each operation should be displayed clearly.

Classes and Important Methods

1. ArrayList Class

`ArrayList` is a resizable-array implementation of the `List` interface.

Methods demonstrated:

- `add()`
- `add(index, element)`
- `get()`
- `set()`
- `remove()`
- `contains()`
- `size()`
- `isEmpty()`
- `clear()`
- `indexOf()`
- `lastIndexOf()`
- `sort()`

2. LinkedList Class

`LinkedList` is a doubly-linked list implementation.

Methods demonstrated:

- `add()`
 - `addFirst()`
 - `addLast()`
 - `get()`
 - `getFirst()`
 - `getLast()`
 - `remove()`
 - `removeFirst()`
 - `removeLast()`
 - `offer()`
 - `poll()`
 - `peek()`
-

3. Vector Class

`Vector` is a synchronized, resizable-array implementation.

Methods demonstrated:

- `add()`
 - `addElement()`
 - `get()`
 - `set()`
 - `remove()`
 - `removeElement()`
 - `size()`
 - `capacity()`
 - `contains()`
-

4. Stack Class

`Stack` follows the **LIFO (Last In, First Out)** principle.

Methods demonstrated:

- `push()`
 - `pop()`
 - `peek()`
 - `empty()`
 - `search()`
-

5. HashSet Class

`HashSet` stores unique elements and does not guarantee insertion order.

Methods demonstrated:

- `add()`
 - `remove()`
 - `contains()`
 - `size()`
 - `isEmpty()`
 - `clear()`
-

6. LinkedHashSet Class

`LinkedHashSet` stores unique elements while maintaining insertion order.

Methods demonstrated:

- `add()`
 - `remove()`
 - `contains()`
 - `size()`
 - `clear()`
-

7. TreeSet Class

`TreeSet` stores unique elements in sorted order.

Methods demonstrated:

- `add()`
 - `remove()`
 - `contains()`
 - `first()`
 - `last()`
 - `higher()`
 - `lower()`
 - `ceiling()`
 - `floor()`
 - `pollFirst()`
 - `pollLast()`
-

8. PriorityQueue Class

`PriorityQueue` processes elements according to their priority.

Methods demonstrated:

- `add()`
- `offer()`
- `peek()`
- `poll()`
- `remove()`
- `contains()`

- `size()`
-

9. ArrayDeque Class

`ArrayDeque` is a resizable-array implementation of the `Deque` interface.

Methods demonstrated:

- `addFirst()`
 - `addLast()`
 - `offerFirst()`
 - `offerLast()`
 - `pollFirst()`
 - `pollLast()`
 - `peekFirst()`
 - `peekLast()`
-

10. HashMap Class

`HashMap` stores key-value pairs without guaranteeing key order.

Methods demonstrated:

- `put()`
 - `get()`
 - `remove()`
 - `containsKey()`
 - `containsValue()`
 - `keySet()`
 - `values()`
 - `entrySet()`
 - `size()`
 - `isEmpty()`
 - `clear()`
 - `getOrDefault()`
-

11. LinkedHashMap Class

`LinkedHashMap` stores key-value pairs while maintaining insertion order.

Methods demonstrated:

- `put()`
 - `get()`
 - `remove()`
 - `containsKey()`
 - `keySet()`
 - `values()`
 - `entrySet()`
-

12. TreeMap Class

`TreeMap` stores key-value pairs with keys sorted in ascending order.

Methods demonstrated:

- `put()`
 - `get()`
 - `remove()`
 - `containsKey()`
 - `containsValue()`
 - `firstKey()`
 - `lastKey()`
 - `higherKey()`
 - `lowerKey()`
 - `ceilingKey()`
 - `floorKey()`
 - `entrySet()`
-

13. Hashtable Class

`Hashtable` is a synchronized key-value collection.

Methods demonstrated:

- `put()`
- `get()`
- `remove()`
- `containsKey()`
- `containsValue()`
- `keys()`
- `elements()`
- `size()`

- isEmpty()

Algorithm / Logic Description

1. Import the `java.util` package.
2. Create an `ArrayList` and demonstrate insertion, retrieval, updating, deletion, searching, sorting, and size operations.
3. Create a `LinkedList` and demonstrate operations at the beginning and end of the list.
4. Create a `Vector` and demonstrate its resizable-array methods.
5. Create a `Stack` and demonstrate LIFO operations using `push()`, `pop()`, `peek()`, and `search()`.
6. Create a `HashSet` and demonstrate unique-element operations.
7. Create a `LinkedHashSet` and demonstrate unique elements while maintaining insertion order.
8. Create a `TreeSet` and demonstrate sorted and navigation operations.
9. Create a `PriorityQueue` and demonstrate priority-based insertion and removal.
10. Create an `ArrayDeque` and demonstrate operations from both ends.
11. Create a `HashMap` and demonstrate key-value operations.
12. Create a `LinkedHashMap` and demonstrate insertion-order key-value operations.
13. Create a `TreeMap` and demonstrate sorted-key and navigation operations.
14. Create a `Hashtable` and demonstrate synchronized key-value operations.
15. Display the result of each collection operation.

Source Code

```
package tasks;

import java.util.*;

public class CollectionFrameworkPart2 {

    public static void main(String[] args) {

        // =====
        // 1. ARRAYLIST
        // =====

        System.out.println("===== ARRAYLIST =====");

        ArrayList<String> list = new ArrayList<>();
```

```

// add()
list.add("Apple");
list.add("Banana");
list.add("Mango");
list.add("Apple");

System.out.println("ArrayList: " + list);

// add(index, element)
list.add(1, "Orange");
System.out.println("After add(1, Orange): " + list);

// get()
System.out.println("Element at index 2: "
    + list.get(2));

// set()
list.set(2, "Grapes");
System.out.println("After set(): " + list);

// remove(index)
list.remove(1);
System.out.println("After remove(1): " + list);

// remove(Object)
list.remove("Apple");
System.out.println("After remove(Apple): " + list);

// contains()
System.out.println("Contains Mango: "
    + list.contains("Mango"));

// size()
System.out.println("Size: " + list.size());

// isEmpty()
System.out.println("Is Empty: "
    + list.isEmpty());

// indexOf()
System.out.println("Index of Apple: "
    + list.indexOf("Apple"));

// lastIndexOf()
System.out.println("Last Index of Apple: "
    + list.lastIndexOf("Apple"));

// sort()
list.sort(Comparator.naturalOrder());
System.out.println("Sorted List: " + list);

// =====
// 2. LINKEDLIST
// =====

```

```

System.out.println("\n===== LINKEDLIST =====");

LinkedList<String> linkedList =
    new LinkedList<>();

linkedList.add("B");
linkedList.add("C");

System.out.println("LinkedList: "
    + linkedList);

linkedList.addFirst("A");
linkedList.addLast("D");

System.out.println("After addFirst/addLast: "
    + linkedList);

System.out.println("Element at index 1: "
    + linkedList.get(1));

System.out.println("First Element: "
    + linkedList.getFirst());

System.out.println("Last Element: "
    + linkedList.getLast());

linkedList.offer("E");
System.out.println("After offer(E): "
    + linkedList);

System.out.println("Peek: "
    + linkedList.peek());

System.out.println("Poll: "
    + linkedList.poll());

linkedList.removeFirst();
linkedList.removeLast();

System.out.println("After removeFirst/removeLast: "
    + linkedList);

// =====
// 3. VECTOR
// =====

System.out.println("\n===== VECTOR =====");

Vector<String> vector = new Vector<>();

vector.add("Red");
vector.add("Blue");

vector.addElement("Green");

System.out.println("Vector: " + vector);

```

```

System.out.println("Element at index 1: "
    + vector.get(1));

vector.set(1, "Yellow");

System.out.println("After set(): "
    + vector);

System.out.println("Contains Red: "
    + vector.contains("Red"));

System.out.println("Size: "
    + vector.size());

System.out.println("Capacity: "
    + vector.capacity());

vector.remove(0);

System.out.println("After remove(0): "
    + vector);

vector.removeElement("Green");

System.out.println("After removeElement(Green): "
    + vector);

// =====
// 4. STACK
// =====

System.out.println("\n==== STACK =====");

Stack<Integer> stack = new Stack<>();

stack.push(10);
stack.push(20);
stack.push(30);

System.out.println("Stack: " + stack);

System.out.println("Peek: "
    + stack.peek());

System.out.println("Search 20: "
    + stack.search(20));

System.out.println("Pop: "
    + stack.pop());

System.out.println("Stack after pop: "
    + stack);

System.out.println("Is Empty: "
    + stack.empty());

```

```

// =====
// 5. HASHSET
// =====

System.out.println("\n==== HASHSET =====");

HashSet<Integer> hashSet =
    new HashSet<>();

hashSet.add(10);
hashSet.add(20);
hashSet.add(30);
hashSet.add(20);

System.out.println("HashSet: "
    + hashSet);

System.out.println("Contains 20: "
    + hashSet.contains(20));

System.out.println("Size: "
    + hashSet.size());

hashSet.remove(10);

System.out.println("After remove(10): "
    + hashSet);

System.out.println("Is Empty: "
    + hashSet.isEmpty());

hashSet.clear();

System.out.println("After clear(): "
    + hashSet);

// =====
// 6. LINKEDHASHSET
// =====

System.out.println("\n==== LINKEDHASHSET =====");

LinkedHashSet<String> linkedHashSet =
    new LinkedHashSet<>();

linkedHashSet.add("Apple");
linkedHashSet.add("Banana");
linkedHashSet.add("Mango");
linkedHashSet.add("Apple");

System.out.println("LinkedHashSet: "
    + linkedHashSet);

System.out.println("Contains Banana: ")

```

```

        + linkedHashSet.contains("Banana"));

System.out.println("Size: "
    + linkedHashSet.size());

linkedHashSet.remove("Banana");

System.out.println("After remove(Banana): "
    + linkedHashSet);

linkedHashSet.clear();

System.out.println("After clear(): "
    + linkedHashSet);

// =====
// 7. TREESET
// =====

System.out.println("\n===== TREESET =====");

TreeSet<Integer> treeSet =
    new TreeSet<>();

treeSet.add(10);
treeSet.add(20);
treeSet.add(30);
treeSet.add(40);
treeSet.add(50);

System.out.println("TreeSet: "
    + treeSet);

System.out.println("First: "
    + treeSet.first());

System.out.println("Last: "
    + treeSet.last());

System.out.println("Higher(30): "
    + treeSet.higher(30));

System.out.println("Lower(30): "
    + treeSet.lower(30));

System.out.println("Ceiling(35): "
    + treeSet.ceiling(35));

System.out.println("Floor(35): "
    + treeSet.floor(35));

System.out.println("Contains 20: "
    + treeSet.contains(20));

treeSet.remove(20);

```

```

System.out.println("After remove(20): "
    + treeSet);

System.out.println("Poll First: "
    + treeSet.pollFirst());

System.out.println("Poll Last: "
    + treeSet.pollLast());

System.out.println("After polling: "
    + treeSet);

// =====
// 8. PRIORITYQUEUE
// =====

System.out.println("\n==== PRIORITYQUEUE =====");

PriorityQueue<Integer> priorityQueue =
    new PriorityQueue<>();

priorityQueue.add(30);
priorityQueue.offer(10);
priorityQueue.add(20);

System.out.println("PriorityQueue: "
    + priorityQueue);

System.out.println("Peek: "
    + priorityQueue.peek());

System.out.println("Contains 20: "
    + priorityQueue.contains(20));

System.out.println("Size: "
    + priorityQueue.size());

System.out.println("Poll: "
    + priorityQueue.poll());

priorityQueue.remove(30);

System.out.println("After remove(30): "
    + priorityQueue);

// =====
// 9. ARRAYDEQUE
// =====

System.out.println("\n==== ARRAYDEQUE =====");

ArrayDeque<String> deque =
    new ArrayDeque<>();

deque.addFirst("B");

```

```

deque.addLast("C");

deque.offerFirst("A");
deque.offerLast("D");

System.out.println("ArrayDeque: "
    + deque);

System.out.println("Peek First: "
    + deque.peekFirst());

System.out.println("Peek Last: "
    + deque.peekLast());

System.out.println("Poll First: "
    + deque.pollFirst());

System.out.println("Poll Last: "
    + deque.pollLast());

System.out.println("ArrayDeque after polling: "
    + deque);

// =====
// 10. HASHMAP
// =====

System.out.println("\n===== HASHMAP =====");

HashMap<Integer, String> hashMap =
    new HashMap<>();

hashMap.put(101, "Rahul");
hashMap.put(102, "Priya");
hashMap.put(103, "Akshaya");

System.out.println("HashMap: "
    + hashMap);

System.out.println("Value for key 102: "
    + hashMap.get(102));

System.out.println("Contains key 101: "
    + hashMap.containsKey(101));

System.out.println("Contains value Rahul: "
    + hashMap.containsValue("Rahul"));

System.out.println("Keys: "
    + hashMap.keySet());

System.out.println("Values: "
    + hashMap.values());

System.out.println("Entries: "
    + hashMap.entrySet());

```

```

System.out.println("Size: "
    + hashMap.size());

System.out.println("Value for key 999: "
    + hashMap.getOrDefault(999, "Not Found"));

hashMap.remove(103);

System.out.println("After remove(103): "
    + hashMap);

System.out.println("Is Empty: "
    + hashMap.isEmpty());

// =====
// 11. LINKEDHASHMAP
// =====

System.out.println("\n==== LINKEDHASHMAP =====");

LinkedHashMap<Integer, String> linkedHashMap =
    new LinkedHashMap<>();

linkedHashMap.put(1, "One");
linkedHashMap.put(2, "Two");
linkedHashMap.put(3, "Three");

System.out.println("LinkedHashMap: "
    + linkedHashMap);

System.out.println("Value for key 2: "
    + linkedHashMap.get(2));

System.out.println("Contains key 1: "
    + linkedHashMap.containsKey(1));

System.out.println("Keys: "
    + linkedHashMap.keySet());

System.out.println("Values: "
    + linkedHashMap.values());

System.out.println("Entries: "
    + linkedHashMap.entrySet());

linkedHashMap.remove(2);

System.out.println("After remove(2): "
    + linkedHashMap);

// =====
// 12. TREEMAP
// =====

```

```

System.out.println("\n===== TREEMAP =====");

TreeMap<Integer, String> treeMap =
    new TreeMap<>();

treeMap.put(10, "Ten");
treeMap.put(20, "Twenty");
treeMap.put(30, "Thirty");
treeMap.put(40, "Forty");
treeMap.put(50, "Fifty");

System.out.println("TreeMap: "
    + treeMap);

System.out.println("Value for key 30: "
    + treeMap.get(30));

System.out.println("Contains key 20: "
    + treeMap.containsKey(20));

System.out.println("Contains value Thirty: "
    + treeMap.containsValue("Thirty"));

System.out.println("First Key: "
    + treeMap.firstKey());

System.out.println("Last Key: "
    + treeMap.lastKey());

System.out.println("Higher Key(30): "
    + treeMap.higherKey(30));

System.out.println("Lower Key(30): "
    + treeMap.lowerKey(30));

System.out.println("Ceiling Key(35): "
    + treeMap.ceilingKey(35));

System.out.println("Floor Key(35): "
    + treeMap.floorKey(35));

System.out.println("Entries: "
    + treeMap.entrySet());

treeMap.remove(20);

System.out.println("After remove(20): "
    + treeMap);

// =====
// 13. HASHTABLE
// =====

System.out.println("\n===== HASHTABLE =====");

Hashtable<Integer, String> hashtable =

```

```

        new Hashtable<>();

hashtable.put(101, "Java");
hashtable.put(102, "Python");
hashtable.put(103, "C++");

System.out.println("Hashtable: "
    + hashtable);

System.out.println("Value for key 102: "
    + hashtable.get(102));

System.out.println("Contains key 101: "
    + hashtable.containsKey(101));

System.out.println("Contains value Java: "
    + hashtable.containsValue("Java"));

System.out.println("Keys:");

Enumeration<Integer> keys =
    hashtable.keys();

while (keys.hasMoreElements()) {
    System.out.println(keys.nextElement());
}

System.out.println("Values:");

Enumeration<String> values =
    hashtable.elements();

while (values.hasMoreElements()) {
    System.out.println(values.nextElement());
}

System.out.println("Size: "
    + hashtable.size());

hashtable.remove(103);

System.out.println("After remove(103): "
    + hashtable);

System.out.println("Is Empty: "
    + hashtable.isEmpty());

System.out.println("\n===== PROGRAM COMPLETED =====");
    }
}

```

Sample Output

```
===== ARRAYLIST =====
ArrayList: [Apple, Banana, Mango, Apple]
After add(1, Orange): [Apple, Orange, Banana, Mango, Apple]
Element at index 2: Banana
After set(): [Apple, Orange, Grapes, Mango, Apple]
After remove(1): [Apple, Grapes, Mango, Apple]
After remove(Apple): [Grapes, Mango, Apple]
Contains Mango: true
Size: 3
Is Empty: false
Index of Apple: 2
Last Index of Apple: 2
Sorted List: [Apple, Grapes, Mango]

===== LINKEDLIST =====
LinkedList: [B, C]
After addFirst/addLast: [A, B, C, D]
Element at index 1: B
First Element: A
Last Element: D
After offer(E): [A, B, C, D, E]
Peek: A
Poll: A
After removeFirst/removeLast: [C, D]

===== VECTOR =====
Vector: [Red, Blue, Green]
Element at index 1: Blue
After set(): [Red, Yellow, Green]
Contains Red: true
Size: 3
Capacity: 10
After remove(0): [Yellow, Green]
After removeElement(Green): [Yellow]

===== STACK =====
Stack: [10, 20, 30]
Peek: 30
Search 20: 2
Pop: 30
Stack after pop: [10, 20]
Is Empty: false

===== HASHSET =====
HashSet: [10, 20, 30]
Contains 20: true
Size: 3
After remove(10): [20, 30]
Is Empty: false
After clear(): []

===== LINKEDHASHSET =====
LinkedHashSet: [Apple, Banana, Mango]
Contains Banana: true
```

```
Size: 3
After remove(Banana): [Apple, Mango]
After clear(): []

===== TREESSET =====
TreeSet: [10, 20, 30, 40, 50]
First: 10
Last: 50
Higher(30): 40
Lower(30): 20
Ceiling(35): 40
Floor(35): 30
Contains 20: true
After remove(20): [10, 30, 40, 50]
Poll First: 10
Poll Last: 50
After polling: [30, 40]

===== PRIORITYQUEUE =====
PriorityQueue: [10, 30, 20]
Peek: 10
Contains 20: true
Size: 3
Poll: 10
After remove(30): [20]

===== ARRAYDEQUE =====
ArrayDeque: [A, B, C, D]
Peek First: A
Peek Last: D
Poll First: A
Poll Last: D
ArrayDeque after polling: [B, C]

===== HASHMAP =====
HashMap: {101=Rahul, 102=Priya, 103=Akshaya}
Value for key 102: Priya
Contains key 101: true
Contains value Rahul: true
Keys: [101, 102, 103]
Values: [Rahul, Priya, Akshaya]
Entries: [101=Rahul, 102=Priya, 103=Akshaya]
Size: 3
Value for key 999: Not Found
After remove(103): {101=Rahul, 102=Priya}
Is Empty: false

===== LINKEDHASHMAP =====
LinkedHashMap: {1=One, 2=Two, 3=Three}
Value for key 2: Two
Contains key 1: true
Keys: [1, 2, 3]
Values: [One, Two, Three]
Entries: [1=One, 2=Two, 3=Three]
After remove(2): {1=One, 3=Three}

===== TREEMAP =====
```

```
TreeMap: {10=Ten, 20=Twenty, 30=Thirty, 40=Forty, 50=Fifty}
Value for key 30: Thirty
Contains key 20: true
Contains value Thirty: true
First Key: 10
Last Key: 50
Higher Key(30): 40
Lower Key(30): 20
Ceiling Key(35): 40
Floor Key(35): 30
Entries: [10=Ten, 20=Twenty, 30=Thirty, 40=Forty, 50=Fifty]
After remove(20): {10=Ten, 30=Thirty, 40=Forty, 50=Fifty}
```

```
===== HASHTABLE =====
Hashtable: {103=C++, 102=Python, 101=Java}
Value for key 102: Python
Contains key 101: true
Contains value Java: true
Keys:
103
102
101
Values:
C++
Python
Java
Size: 3
After remove(103): {102=Python, 101=Java}
Is Empty: false
```

```
===== PROGRAM COMPLETED =====
```

Note About Output Order

HashSet, HashMap, and Hashtable do **not guarantee iteration order**, so their elements may appear in a different order when the program is executed. The `LinkedHashSet` and `LinkedHashMap` examples maintain insertion order, while `TreeSet` and `TreeMap` maintain sorted order.
